Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Implementasi Algoritma Local Search pada Penyusunan Konfigurasi Paket di dalam Truk



Disusun oleh:

13524073 Keisha Rizka Syofyani

13524091 Vara Azzara Ramli Pulukadang

13524109 Helena Kristela Sarhawa

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **2**
Bab 1
Deskripsi Persoalan **3**
Bab 2
Pembahasan **4**
2.1 Representasi Objek dan State 4
2.2 Objective Function dan Constraint 4
2.3 Representasi Neighbor 5
2.4 Implementasi Algoritma Hill Climbing 6
2.5 Implementasi Algoritma Simulated Annealing 10
2.6 Implementasi Algoritma Genetic 16
2.7 Implementasi Bonus Multi-Truk 20
2.8 Implementasi Bonus Objective Function 20
2.9 Hasil Eksperimen dan Analisis 21
2.9.1 Pengaturan Eksperimen 21
2.9.2 Hasil Hill-Climbing 21
2.9.3 Hasil Simulated Annealing 23
2.9.4 Hasil Genetic Algorithm 24
2.9.5 Analisis Perbandingan Antaralgoritma 27
2.9.6 Analisis Pengaruh Iterasi dan Populasi Genetic Algorithm 31
Bab 3
Kesimpulan dan Saran **38**
3.1 Kesimpulan 38
3.2 Saran 38
Pembagian Tugas **39**
Lampiran **40**
Referensi **41**
Lampiran Form Penggunaan Generative AI **42**

Bab 1
Deskripsi Persoalan

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk ujicoba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

Tugas ini bertujuan untuk menentukan konfigurasi penyusunan paket-paket pada beberapa truk dengan memaksimalkan value yang dimiliki setiap paket yang bisa disusun serta memastikan tidak melanggar constraint yang berlaku. Oleh karena pencarian konfigurasi terbaik secara menyeluruh sangat mahal, sehingga permasalahan ini diselesaikan dengan algoritma *local se*arch. Adapun algoritma local search yang digunakan terdiri atas :

1. Hill-climbing
    a. Steepest Ascent
    b. Sideways Move
    c. Random Restart
    d. Stochastic
2. Simulated Annealing
3. Genetic Algorithm

Paket-paket yang harus disusun berbentuk kubus/balok yang ukurannya berbeda-beda satu sama lain dan tidak semua harus dimasukkan ke dalam truk. Koordinat posisi paket direpresentasikan dengan bilangan bulat (x, y, z). Inisialisasi state dilakukan dengan random dan

Penyusunan dan pemosisian paket dapat dilakukan dengan menukar dua paket di dalam maupun di luar truk, memindahkan posisi paket, dan juga melakukan rotasi 90° terhadap sumbu x, y, atau z. Representasi solusi (final state) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Hasil dan visualisasi akan menunjukkan bagaimana keadaan semua paket yang di dalam maupun yang di luar, bagaimana keadaan peti kemas pada truk, dan berapa value pada state tersebut. Adapun constarint dalam penyusunan paket di dalam truk adalah sebagai berikut.

1. Paket tidak bisa melayang
2. Paket menembus dinding peti kemas
3. Barang pecah-belah tidak boleh di bawah paket lain
4. Barang pecah-belah tidak boleh menjadi support
5. Truk tidak boleh mengangkut melebihi kapasitasnya

Bab 2
Pembahasan

2.1 Representasi Objek dan State

State digunakan untuk memetakan seluruh paket yang ada, baik di dalam maupun di luar truk. Implementasi state terdapat di dalam file models.py. State diimplementasikan dengan tiga kelas utama, yakni

Kelas	Atribut	Keterangan
Package	*id*	Identifier paket
	width, length, height	Ukuran paket
	value	Nilai harga sebuah paket (satuan unit)
	weight	Berat paket (satuan unit berat)
	eta	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
	position	Posisi paket pada truk berdasarkan sumbu x, y, z dari sudut kiri, bawah, belakang truk. Bernilai None jika paket berada di luar truk.
	orientation	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu z.
Truck	*width, length, height*	Ukuran sebuah peti kemas pada truk
	max_capacity	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)
State	*truck*	State truck
	packages	State list semua paket yang diinisialisasi

2.2 Objective Function dan Constraint

Sesuai dengan spesifikasi tugas, objective utama dari kasus ini adalah memaksimalkan peti kemas pada truk dengan memastikan state memenuhi constraint yang ada.

- Implementasi Objective Function

  Implementasi objective function terdapat pada file objective.py. Objective function dihitung dengan menjumlahkan semua value paket yang berhasil

diletakkan pada truk menggunakan fungsi property pada State, yakni *total_value()*. Apabila tidak ada paket yang berhasil dimasukkan ke truk, maka akan mengembalikan nilai tidak valid default (INVALID_VALUE)

- Implementasi Constraint

  Implementasi constraint terdapat pada file validate.py. Fungsi is_valid() digunakan untuk memastikan setiap kondisi state memenuhi semua constraint yang ada dengan beberapa helper function sesuai dengan kondisi yang harus dipenuhi, yang meliputi:

Constraint	**Helper Function**	**Deskripsi Helper Function**
Tidak menemnus dinding	*is_fit_in_truck()*	Memastikan posisi paket x,y,z >= 0 dan (x,y,z) + w paket <= (w, l, h) truk
Tidak overlap	*is_overlap()*	Mengecek adanya overlap pada dua paket, yakni ketika interval posisi x, y, dan z beririsan
Tidak melayang	*is_melayang()*	Memastikan paket tidak melayang, hanya boleh diposisikan di lantai truk (z = 0) atau tepat berhimpitan di atas paket lain
Pecah belah tidak boleh menjadi support	*is_support()*	Melakukan pengecekkan apakah terdapat paket yang berhimpit pada sebuah paket
Kapasitas berat	*State.remaining_weight()*	Melakukan pengecekkan sisa berat paket yang diperbolehkan untuk masuk ke truk sesuai maximum kapasitas truk (max_capacity)

2.3 Representasi Neighbor

Fungsi neighbor berfungsi untuk membangkitkan *state* tetangga dari *state* saat ini. Terdapat tiga jenis langkah atau *move* yang dapat dilakukan.

- *Swap*

  Dengan *swap*, posisi dua paket dapat ditukar. Paket pertama dipilih dari paket yang berada di dalam truk, sedangkan paket kedua dipilih dari semua paket lain. Jika paket kedua berada di luar truk, paket pertama akan dikeluarkan dan paket kedua dimasukkan pada posisi yang sama.

- *Move*

  Move berguna untuk memindahkan satu paket ke koordinat lain. Terdapat tiga variasi yang dapat dipilih dengan peluang yang sama yakni

  1. *take out*: paket di dalam truk dikeluarkan,
  2. *shift*: paket di dalam truk digeser satu unit pada sumbu x atau y, dan

3. *relocate*: paket (di dalam atau luar truk) ditempatkan pada koordinat (x, y) acak di dalam truk.

Paket yang berada di luar truk hanya bisa *relocate*.

- *Rotate*

  Dengan *rotate*, satu paket dapat dirotasi 90° terhadap sumbu x, y, atau z. Rotasi yang tidak mengubah ukuran efektif paket, misalnya pada paket kubus, akan dilewati karena tidak menghasilkan *state* yang berbeda secara geometris.

Saat neighbor dibangkitkan, jenis langkah atau *move* akan dipilih secara acak dengan peluang yang sama. Neighbor yang melanggar *constraint* seperti melayang, tumpang tindih, menembus dinding truk, paket pecah belah menjadi penopang, atau melebih kapasitas berat akan dibuang dengan fungsi is_valid lalu dicoba kembali hingga maksimal seratus kali. Dengan cara seperti itu, *state* yang diproses oleh seluruh algoritma *local search* selalu valid.

Pada langkah *shift, relocate,* dan *rotate*, koordinat z tidak dipilih secara acak. Koordinat z dihitung menggunakan heuristik drop_z, yakni heuristik yang menyatakan bahwa paket seperti dijatuhkan ke permukaan tertinggi yang berada di bawahnya. Nilai z adalah puncak tertinggi dari paket lain yang alasnya beririsan dengan alas paket pada bidang xy, atau nol jika tidak ada. Maka dari itu, paket selalu bertumpu pada lantai truk atau pada paket lain sehingga tidak melayang dan tidak menembus paket di bawahnya.

Untuk kasus multi-truk, setiap paket menyimpan atribut truck_id yang menyatakan truk tempat paket berada. Langkah *swap* akan menukar posisi sekaligus truck_id kedua paket, sehingga paket dapat berpindah antartruk. Langkah *relocate* memilih truk tujuan secara acak lalu memilih koordinat (x, y) acak di dalam truk tersebut. Heuristik drop_z dan seluruh pengecekan *constraint* dihitung untuk setiap truk sehingga paket hanya bertumpu pada paket lain di truk yang sama.

Ukuran ruang neighbor sendiri sangat besar. Untuk n paket pada truk berukuran W x L,

- *swap* = hingga n(n-1)/2 pasangan,
- *relocate* = hingga (W-dx+1)(L-dy+1) posisi untuk setiap paket, dengan dx dan dy sebagai ukuran alas paket, dan
- *rotate* = tiga kemungkinan untuk setiap paket.

2.4 Implementasi Algoritma Hill Climbing

Hill Climbing adalah algoritma local search dengan analogi mendaki Gunung Everest dalam kabut tebal dengan amnesia. Pembangkitan state dilakukan secara acak dan terus bergerak ke arah nilai yang lebih tinggi dan berhenti ketika tidak ada neighbor yang lebih baik lagi. Algoritma Hill Climbing ini diimplementasikan pada file hill_climbing.py. Terdapat beberapa varian algoritma Hill Climbing berdasarkan cara penentuan neighbornya, yakni :

1) Steepest Ascent

   Merupakan algoritma utama Hill Climbing, yang mengambil *neighbor* yang memiliki nilai lebih besar dari *current*. Iterasi pencarian akan berhenti ketika nilai neighbor lebih kecil atau sama dengan *current*, atau jumlah iterasi maksimum sudah tercapai.

```
def steepest_ascent(truck : Truck, packages : List[Package],
max_iteration = MAX_ITERATION)  :
    # Inisialisasi state
```

```
    initial_state = generate(truck, packages)

    current_state = initial_state
    current_value = objective(current_state)

    # Mulai timer
    start_time = time()

     # Membangkitkan tetangga dan memilih neighbor dengan value
>= current state
    for i in range(max_iteration) :
            neighbors = get_neighbor_sample(current_state, n =
NUM_NEIGHBOR)

        if not neighbors : break

                        best_neighbor,  best_neighbor_value  =
get_best_state(neighbors)

        if(best_neighbor_value <= current_value) :
            break

        current_state = best_neighbor
        current_value = best_neighbor_value

    end_time = time()

    duration = end_time - start_time

         return  i,  duration,  initial_state,  current_state,
current_value
```

Initial state adalah posisi paket di dalam truk secara random yang disimpan pada variabel *initial_state* yang sekaligus menjadi current state saat awal mulai proses iterasi. Total nilai value paket di awal juga disimpan pada variabel *current_value*. Iterasi pencarian akan dilakukan dengan maksimum jumlah iterasi tertentu. *Neighbor* akan disimpan pada variabel neighbor berdasarkan fungsi pembangkit *neighbor* (*get_neighbor_sample()*) dan mencari *neighbor* dengan *value* paling tinggi menggunakan fungsi *helper get_best_state()*. *Current* state akan berpindah ke *neighbor* ketika *value neighbor* > *value current*. Iterasi akan terus dilakukan hingga hanya mendapatkan *neighbor* yang memiliki *value* < *value current*.

2) Sideways Move

Merupakan algoritma Hill Climbing yang mengambil *neighbor* yang memiliki nilai lebih besar atau sama dengan *current*. Pada algoritma ini diperbolehkan state berada di *shoulder* (*current* = *neighbor*) untuk jumlah maksimum kondisi sideways move secara berurut tertentu (MAXIMUM_SIDEWAYS_MOVE). Iterasi pencarian akan berhenti hanya ketika nilai *neighbor* lebih kecil dari *current*, mencapai batasan maksimum sideways move, atau jumlah iterasi maksimum sudah tercapai.

```
def sideways_move(truck : Truck, packages : List[Package], max_iteration = MAX_ITERATION, max_sideways_move = MAXIMUM_SIDEWAYS_MOVE) :
    # Inisialisasi state
    initial_state = generate(truck, packages)

    current_state = initial_state
    current_value = objective(current_state)

    sideways_count = 0

    # Mulai timer
    start_time = time()

     # Membangkitkan tetangga dan memilih neighbor dengan value >= current state
    for i in range(max_iteration) :
            neighbors = get_neighbor_sample(current_state, n = NUM_NEIGHBOR)

        if not neighbors : break

                        best_neighbor, best_neighbor_value = get_best_state(neighbors)

        if(best_neighbor_value < current_value) :
            break

        # Kurangi Jatah Batasan Sideways Move (di shoulder)
        if(best_neighbor_value == current_value) :
            if(sideways_count >= max_sideways_move) :
                break

            sideways_count += 1

        else :
            sideways_count = 0

        current_state = best_neighbor
        current_value = best_neighbor_value

    end_time = time()

    duration = end_time - start_time

         return i, duration, initial_state, current_state, current_value
```

Inisialisasi *state* dan *neighbor* sama seperti halnya pada steepest ascent. Pada algoritma ini, *current state* akan berpindah ke *neighbor* ketika *value neighbor* >= *value current*, dengan syarat terjadinya kondisi *value neighbor* = *value current* itu hanya maksimum sebanyak MAXIMUM_SIDEWAYS_MOVE

secara beruturut-turut. Iterasi akan terus dilakukan hingga hanya mendapatkan *neighbor* yang memiliki *value* < *value current*.

3) Random Restart

Merupakan algoritma Hill Climbing yang membangkitkan *neighbor* secara terus menerus dan memilih *neighbor* berdasarkan pembangkitan steepest ascent degan *value* terbaik dengan harapan mendapatkan nilai maksimum global dari beberapa kali percobaan tersebut.

```
def random_restart(truck : Truck, packages : List[Package], max_restart) :
    total_iteration = 0
    initial_state = None

    best_state = None
    best_value = INVALID_VALUE

    start_time = time()

    for i in range(max_restart) :
                    it, _, init, finding, finding_value = steepest_ascent(truck, packages)

        total_iteration += it

        if(not initial_state) :
            initial_state = init

        if(finding_value > best_value) :
            best_state = finding
            best_value = finding_value

    end_time = time()

    duration = end_time - start_time

          return total_iteration, duration, initial_state, best_state, best_value
```

Pencarian dilakukan berulang kali dengan state awal yang berbeda. Pencariannya menggunakan konsep steepest ascent, namun memperbolehkan dilakukannya perulangan pencarian berulang kali hingga mencapai *max_restart*. State yang diambil adalah state yang memiliki nilai tertinggi setiap iterasinya dan disimpan untuk dibandingkan untuk setiap restart. Algoritma ini melengkapi algoritma steepest ascent biasa sehingga pencarian yang dilakukan dapat lebih maksimal.

4) Stochastic

Merupakan algoritma Hill Climbing yang membangkitkan suksesor secara random dan memilihnya jika nilainya lebih baik dari *current*.

```
def stochastic(truck : Truck, packages : List[Package],
max_iteration = MAX_ITERATION) :
    # Inisialisasi state
    initial_state = generate(truck, packages)

    current_state = initial_state
    current_value = objective(current_state)

    # Mulai timer
    start_time = time()

    for i in range(max_iteration) :
            neighbors = get_neighbor_sample(current_state, n =
NUM_NEIGHBOR)

        if(not neighbors) : break

        random_neighbor = random.choice(neighbors)

        if(not random_neighbor) : break

        random_neighbor_value = objective(random_neighbor)

        # Ganti current state kalau neighbor random lebih baik
        if(random_neighbor_value > current_value) :
            current_state =  random_neighbor
            current_value = random_neighbor_value

    end_time = time()

    duration = end_time - start_time

         return i, duration, initial_state, current_state,
current_value
```

Inisialisasi state sama seperti halnya pada steepest ascent. *Neighbor* akan dipilih seacara random dengan random.choice(). Pada algoritma ini, current state akan berpindah ke neighbor ketika value *neighbor* yang didapatkan > *value current* berdasarkan pencarian random tersebut. Iterasi ini akan terus diulang hingga mencapai *max_iteration*.

2.5 Implementasi Algoritma Simulated Annealing

Simulated Annealing adalah algoritma varian dari stochastic hill-climbing yang mengizinkan sebagian langkah ke *state* yang lebih buruk agar pencarian bisa keluar dari *local optimum*. Implementasi Simulated Annealing diterapkan dengan langkah-langkah berikut.

1. Dimulai dari *state* awal acak, hasil dari generate()
2. Pada setiap iterasi, pilih satu neighbor acak dan hitung ΔE = value(neighbor) − value(current)

3. Jika ΔE > 0, pindah ke neighbor
4. Jika ΔE ≤ 0, pindah ke neighbor dengan probabilitas e^(ΔE/T), yaitu jika e^(ΔE/T) > random(0, 1)
5. Turunkan temperatur T secara bertahap

State yang dikembalikan adalah *state current* pada saat pencarian berhenti. *State* terbaik yang pernah ditemui (*best*) tetap dicatat sebagai informasi tambahan untuk analisis.

Berikut terlampir tabel parameter yang digunakan. Seluruh nilai parameter diambil berdasarkan studi eksperimental Johnson dkk. [2]

Parameter	Nilai	Arti
`INITPROB`	0,4	Peluang awal diterimanya *move* yang memperburuk
`TEMPFACTOR (r)`	0,95	Rasio pendinginan geometrik T ← r · T
`SIZEFACTOR`	16	Panjang satu level temperatur L = 16 · N
`MINPERCENT`	2%	Batas persentase penerimaan untuk kondisi *frozen*
Batas *Frozen*	5	Banyak level berturut-turut untuk berhenti

Implementasi pun menggunakan *cooling schedule* geometrik. Temperatur dipertahankan selama L iterasi (satu level temperatur) kemudian diturunkan dengan faktor pendinginan r seperti T(k+1) = r · T(k) dengan r = 0,95. Panjang satu level temperatur ditentukan oleh L = SIZEFACTOR x N = 16N dengan N adalah ukuran neighborhood. N didefinisikan sebagai banyaknya paket karena satu langkah neighbor mengubah satu paket atau sepasang paket jika menggunakan *swap*.

T_0 sebagai temperatur awal ditentukan berdasarkan *acceptance ratio of bad moves* yang mengikuti sumber [6]. T_0 dipilih sehingga rata-rata peluang diterimanya langkah yang memperburuk pada *state* awal mendekati nilai INITPROB

$$\frac{1}{|W|} \sum_{i \in W} e^{-|\Delta E_i|/T_0} \approx 0,4$$

dengan W adalah himpunan langkah yang memperburuk (ΔE < 0) dari L neighbor yang dibangkitkan secara acak dari *state* awal. Oleh karena ruas kiri persamaan naik secara monoton terhadap T_0, nilai T_0 dicari dengan metode *bisection* pada skala logaritmik. Persentase penerimaan hanya dihitung dari langkah yang memperburuk karena sebagian besar langkah pada persoalan ini bernilai ΔE = 0, misalnya pada rotasi atau pergeseran paket yang tidak mengubah total *value,* dan langkah seperti itu selalu diterima. Jika semua langkah ikut dihitung, persentase penerimaan tidak akan pernah turun ke nilai target. Apabila tidak ditemukan satu pun langkah yang memperburuk pada sampel awal,

misalnya ketika *state* awal tidak memiliki paket di dalam truk, T_0 diisi dengan nilai cadangan 1.

Pada pendinginan geometrik, temperatur tidak pernah mencapai T = 0 sehingga kondisi berhenti T = 0 tidak dapat dipakai secara langsung. Maka dari itu pada implementasi ini, digunakan kriteria berhenti (*frozen*) dari sumber [2] yakni sebagai berikut.

- Sebuah *counter* bertambah satu setiap kali satu level temperatur selesai dengan persentase penerimaan langkah yang memperburuk < MINPERCENT
- *Counter* di-*reset* ke nol setiap kali ditemukan *state* yang lebih baik daripada *state* terbaik sebelumnya
- Pencarian berhenti ketika *counter* mencapai 5

Dengan kriteria di atas, pencarian akan berhenti setelah temperatur cukup rendah sehingga hampir tidak ada langkah buruk yang diterima dan tidak ada perbaikan selama lima level temperatur berturut-turut.

Pada implementasi ini, satu kejadian *stuck* dihitung apabila satu level penuh temperatur berlalu tanpa ditemukannya *state* yang lebih baik daripada *state* terbaik sebelumnya. Frekuensi *stuck* adalah jumlah level temperatur yang memenuhi kondisi tersebut selama satu kali pencarian.

Berikut adalah daftar fungsi yang digunakan untuk algoritma Simulated Annealing.

Fungsi	Keterangan
`find_initial_temperature(...)`	Menentukan T0 berdasarkan INITPROB
`simulated_annealing(initial_state, ...)`	Algoritma Simulated Annealing yang mengembalikan *state* awal dan akhir, nilai akhir, riwayat nilai *objective*, riwayat e^(ΔE/T), frekuensi *stuck*, banyak iterasi, dan durasi
`run_experiment(...)` `(experiment_runner.py)`	Menjalankan algoritma beberapa kali, lalu menyimpan *state*, hasil, log, dan plot ke folder output/

```
# sumber parameter dari Johnson dkk. (1989), Operations Research
37(6), dan Ben-Ameur (2004), COAP 29(3)
import math
import random
import time
from neighbor import get_neighbor_with_info
from objective import objective
from validator import is_valid
```

```
INITPROB = 0.4
TEMPFACTOR = 0.95
SIZEFACTOR = 16
MINPERCENT = 0.02
FROZEN_LIMIT = 5

def _acceptance_fraction(worse, T):
    return sum(math.exp(-w / T) for w in worse) / len(worse)

def find_initial_temperature(state, objective_fn, is_valid_fn, rng,
n_samples, init_prob=INITPROB, iterations=60):
    base = objective_fn(state)
    worse = []
    for _ in range(n_samples):
            candidate, name = get_neighbor_with_info(state, rng,
is_valid_fn)
        if name is None:
            continue
        d = objective_fn(candidate) - base
        if d < 0:
            worse.append(-d)
    if not worse:
        return 1.0
    scale = sum(worse) / len(worse)
    lo, hi = scale * 1e-6, scale * 1e6
    for _ in range(iterations):
        mid = math.sqrt(lo * hi)
        if _acceptance_fraction(worse, mid) < init_prob:
            lo = mid
        else:
            hi = mid
    return math.sqrt(lo * hi)

def simulated_annealing(initial_state, objective_fn=objective,
is_valid_fn=is_valid, init_prob=INITPROB, temp_factor=TEMPFACTOR,
size_factor=SIZEFACTOR, min_percent=MINPERCENT,
frozen_limit=FROZEN_LIMIT, neighborhood_size=None, T0=None,
max_iter=1_000_000, seed=None, verbose=False):
    rng = random.Random(seed) if seed is not None else random
    start = time.perf_counter()
```

```
    N = neighborhood_size or len(initial_state.packages)
    L = max(1, int(size_factor * N))
    if T0 is None:
          T0 = find_initial_temperature(initial_state, objective_fn,
is_valid_fn, rng, n_samples=L, init_prob=init_prob)
    current = initial_state
    current_value = objective_fn(current)
    best, best_value = current, current_value
    history = [current_value]
    best_history = [best_value]
    prob_history, temp_history, acceptance_rate_history = [], [], []
    stuck_count, stuck_iterations = 0, []
    frozen_counter = 0
    accepted = 0
    move_counts = {"swap": 0, "move": 0, "rotate": 0, "none": 0}
    T = T0
    iteration = 0
    temperatures = 0
    stopped_by = "frozen"
    while frozen_counter < frozen_limit:
        temperatures += 1
        tried_worse, accepted_worse = 0, 0
        new_champion = False
        for _ in range(L):
            if iteration >= max_iter:
                break
            iteration += 1
              candidate, move_name = get_neighbor_with_info(current,
rng, is_valid_fn)
            move_counts[move_name or "none"] += 1
            candidate_value = objective_fn(candidate)
            delta = candidate_value - current_value
            prob = 1.0 if delta > 0 else math.exp(delta / T)
            is_accepted = delta > 0 or rng.random() < prob
            if is_accepted:
                current, current_value = candidate, candidate_value
                if move_name is not None:
                    accepted += 1
            if delta < 0:
                tried_worse += 1
```

```
                accepted_worse += int(is_accepted)
            if current_value > best_value:
                best, best_value = current, current_value
                new_champion = True
                frozen_counter = 0
            history.append(current_value)
            best_history.append(best_value)
            prob_history.append(prob)
            temp_history.append(T)
        rate = accepted_worse / tried_worse if tried_worse else 0.0
        acceptance_rate_history.append(rate)
        if not new_champion:
            stuck_count += 1
            stuck_iterations.append(iteration)
        if rate <= min_percent:
            frozen_counter += 1
        if verbose:
                print(f"[SA] suhu ke-{temperatures:3d}   T={T:10.4f}
accept={rate:6.2%}   current={current_value:8}   best={best_value:8}
frozen={frozen_counter}")
        if iteration >= max_iter:
            stopped_by = "max_iter"
            break
        T *= temp_factor
    duration = time.perf_counter() - start

    return {
        "algorithm": "Simulated Annealing",
        "initial_state": initial_state,
        "initial_value": history[0],
        "final_state": current,
        "final_value": current_value,
        "best_state": best,
        "best_value": best_value,
        "history": history,
        "best_history": best_history,
        "prob_history": prob_history,
        "temp_history": temp_history,
        "acceptance_rate_history": acceptance_rate_history,
        "stuck_count": stuck_count,
```

```
        "stuck_iterations": stuck_iterations,
        "iterations": iteration,
        "temperatures": temperatures,
        "accepted_moves": accepted,
        "move_counts": move_counts,
        "stopped_by": stopped_by,
        "duration": duration,
        "params": {
                  "T0": T0, "init_prob": init_prob, "temp_factor":
temp_factor,
            "size_factor": size_factor, "neighborhood_size": N,
            "temp_length": L, "min_percent": min_percent,
                "frozen_limit": frozen_limit, "max_iter": max_iter,
"seed": seed,
        },
    }
```

2.6 Implementasi Algoritma Genetic

Genetic Algorithm mempertahankan sekumpulan *state* yang disebut populasi. Populasi diperbarui lewat empat tahap, yaitu seleksi, *crossover*, mutasi, dan pembentukan generasi baru. Implementasi Genetic Algorithm terdapat pada file genetic.py.

Satu individu adalah satu objek State yang berisi truk dan *state* setiap paket yang ada pada individu tersebut. *State* setiap paket mencakup orientasi dan posisi paket.

```
class State :
    truck : Truck
    packages : List[Package]
```

Urutan paket pada daftar packages selalu sama untuk semua individu, artinya paket pada indeks i adalah paket dengan identitas sama, namun bisa memiliki orientasi dan posisi yang berbeda. Dengan begitu, *crossover* dapat dilakukan dengan indeks. Fungsi fitness menggunakan *objective function* yang sama dengan algoritma lain dengan nilai minimal 0 karena proses seleksi harus menggunakan nilai nonnegatif.

Alur algoritmanya sebagai berikut.

1. Persiapan:
    a. Bangkitkan `population_size` individu secara acak dengan `generate(truck, packages)`. Paket dipastikan valid.
    b. Hitung *fitness* setiap individu, yaitu total *value* paket di dalam truk.
    c. Individu terbaik dicatat sebagai `initial_state`, nilai *fitness* maksimum dan rata-rata *fitness* pada populasi tersebut juga disimpan.
2. Setiap generasi menghasilkan satu populasi baru dengan ukuran yang sama. Ulangi langkah di bawah hingga populasi penuh:
    a. Seleksi: pilih dua individu sebagai orang tua dengan *roulette wheel*. Individu dengan *fitness* lebih tinggi punya peluang terpilih lebih besar.

b. *Crossover*: gabungkan kedua orang tua menjadi satu anak. Paket pada indeks sebelum titik potong mengikuti orang tua pertama, sisanya mengikuti orang tua kedua.
c. *Repair*: perbaiki anak agar valid untuk paket yang bertabrakan, melayang, bertumpu pada paket pecah belah, atau melebihi kapasitas truk.
d. Mutasi: denga peluang `mutation_rate`, ubah anak dengan satu langkah `neighbor` (swap, move, atau rotate).

3. Setelah populasi baru penuh, populasi baru menggantikan populasi lama. Hitung nilai *fitness* semua individu, simpan maksimum dan rata-ratanya.
4. Setelah semua generasi selesai, kembalikan individu dengan *fitness* tertinggi pada populasi terakhir sebagai `final_state`.

```
def genetic_algorithm(truck, packages, population_size=50,
generations=100,
                              mutation_rate=0.1, elitism=False,
seed=None):
    if population_size < 2:
        raise ValueError("population_size minimal 2")
    if generations < 0:
        raise ValueError("generations tidak boleh negatif")
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()

        population = [generate(truck, packages) for _ in
range(population_size)]
    fitness = [max(0, objective(s)) for s in population]

                best_idx     =     max(range(population_size),
key=fitness.__getitem__)
    initial_state = clone_state(population[best_idx])
    history = [(max(fitness), sum(fitness) / population_size)]

    for _ in range(generations):
        new_population = []
        if elitism:
                        best_idx  =  max(range(population_size),
key=fitness.__getitem__)

new_population.append(clone_state(population[best_idx]))
```

```
        pick = make_roulette(population, fitness)
        while len(new_population) < population_size:
            child = crossover(pick(), pick())
            if random.random() < mutation_rate:
                child = mutate(child)
            new_population.append(child)

        population = new_population
        fitness = [max(0, objective(s)) for s in population]
                 history.append((max(fitness),  sum(fitness)  /
population_size))

                 best_idx     =     max(range(population_size),
key=fitness.__getitem__)
    duration = time.perf_counter() - start
            return   generations,   duration,   initial_state,
population[best_idx], fitness[best_idx], history
```

Fungsi mengembalikan tuple `(generations, duration, initial_state, best_state, best_fitness, history)`. `history` berisi pasangan (*fitness* maksimum, *fitness* rata-rata) untuk setiap generasi, termasuk generasi ke-0, sehingga panjangnya `history` adalah generations + 1. Data ini langsung dipakai untuk plot sesuai ketentuan spesifikasi.

Daftar parameter yang digunakan sebagai berikut.

Parameter	Nilai	Keterangan
`population_size`	50	Jumlah individu per generasi (minimal 2). Diubah pada eksperimen.
`generations`	100	Banyak iterasi (generasi), tidak boleh negatif. Diubah pada eksperimen.
`mutation_rate`	0.1	Peluang sebuah anak dimutasi
`elitism`	`False`	Jika `True`, individu terbaik disalin ke generasi berikutnya
`seed`	`None`	Seed *random* agar

		eksperimen dapat direproduksi

Sementara daftar fungsi yang digunakan sebagai berikut.

Fungsi	Keterangan
`clone_state(state)`	Membuat salinan *state* (setiap paket disalin) agar *state* awal tidak berubah
`_can_add(cand, accepted, weight, truck)`	Mengecek apakah sebuah paket dapat ditambahkan ke himpunan paket yang sudah diterima tanpa melanggar *constraint*
`repair(state)`	Memperbaiki individu yang melanggar *constraint*
`make_roulette(population, fitness)`	Membuat fungsi pemilih orang tua dengan *roulette wheel*
`crossover(parent_x, parent_y)`	*One-point crossover* diikuti repair
`mutate(state)`	Mutasi dengan memanggil `get_neighbor`
`genetic_algorithm(...)`	Fungsi utama GA

Seleksi memilih orang tua dengan metode *roulette wheel*. Individu dengan fitness lebih tinggi memiliki peluang terpilih lebih besar, sebanding dengan nilai fitness-nya. Jika semua individu bernilai 0, algoritma memilih orang tua secara acak dan seragam.

Crossover menggabungkan dua orang tua menjadi satu anak dengan metode *one-point crossover*. Algoritma memilih satu titik potong secara acak. Paket sebelum titik potong mengambil posisi dan orientasi dari orang tua pertama, sedangkan paket sesudahnya mengambil dari orang tua kedua. Penggabungan ini dapat menghasilkan anak yang tidak valid, misalnya karena dua paket bertabrakan atau total berat melebihi kapasitas. Oleh karena itu, setiap anak diperbaiki dengan fungsi `repair` sebelum masuk ke populasi.

Fungsi `repair` memperbaiki anak secara *greedy*. Fungsi ini mengurutkan paket di dalam truk dari koordinat z terendah, lalu memeriksa tiap paket satu per satu. Pengurutan ini membuat paket penopang diperiksa lebih dulu daripada paket di atasnya, sesuai pendekatan *bottom-left* pada permasalahan *packing* [3][4]. Sebuah paket diterima jika berat total tidak melebihi kapasitas, paket berada di dalam truk, tidak bertabrakan dengan paket lain, tidak melayang, dan tidak bertumpu pada paket pecah belah. Paket yang gagal diperiksa dikeluarkan dari truk dengan mengubah posisinya menjadi `None`. Paket tersebut tetap ada di dalam `State` dan dapat masuk kembali melalui *crossover* atau mutasi berikutnya.

Pada mutasi, setiap anak dimutasi dengan peluang `mutation_rate`. Mutasi dilakukan dengan satu langkah *neighbor* (swap, move, atau rotate) memakai fungsi `get_neighbor` yang sama dengan algoritma lain. Karena `get_neighbor` hanya menghasilkan *state* valid, hasil mutasi selalu valid.

Pada pengaturan `elitism=True`, algoritma menyalin individu terbaik ke generasi berikutnya tanpa perubahan. Cara ini menjaga fitness maksimum agar tidak turun antar generasi. Pada pengaturan default (`elitism=False`), individu terbaik dapat hilang karena seleksi, *crossover*, atau mutasi yang bersifat acak. Pengaturan ini dipilih supaya implementasi fokus pada salindia kuliah.

2.7 Implementasi Bonus Multi-Truk

Pada bonus multi-truk, paket dapat dimasukkan ke salah satu dari beberapa truk yang tersedia, dan setiap truk dapat memiliki ukuran serta kapasitas yang berbeda. Tujuan pencarian adalah memaksimalkan total *value* paket yang diangkut oleh seluruh truk. Persoalan satu truk diperlakukan sebagai kasus khusus dengan satu truk sehingga satu implementasi yang sama dapat menangani kedua kasus. Program akan membaca daftar truk dari key Trucks pada *file input*. Jika input hanya memiliki key Truck, truk tersebut dibaca sebagai daftar dengan satu truk sehingga format *input* satu truk tetap dapat digunakan.

Berikut terlampir perubahan yang dilakukan terkait multi-truk.

Komponen	Perubahan
Representasi state	State menyimpan daftar truk (trucks). Setiap Package memiliki atribut truck_id yang menyatakan indeks truk tempat paket berada.
Pembacaan input dan output	Menerima key Truck maupun Trucks. State yang disimpan ke JSON memuat daftar truk dan truck_id setiap paket.
Constraint	Seluruh *constraint* diperiksa per truk. Kapasitas berat dihitung dari paket di truk yang sama, dan pengecekan tumpang tindih, melayang, serta paket pecah belah hanya dilakukan antarpaket di truk yang sama.
Initial state	Setiap paket yang dimasukkan diberi truk tujuan secara acak, lalu ditempatkan pada koordinat acak di truk tersebut dengan heuristik drop_z.
Neighbor	*Swap* menukar posisi sekaligus truck_id dua paket. *Relocate* memilih truk tujuan secara acak, sehingga paket dapat

	berpindah antartruk. Heuristik drop_z dihitung per truk.
Genetic Algorithm	Fungsi *repair* memeriksa dan memperbaiki individu hasil *crossover* secara terpisah untuk setiap truk.
Visualisasi	Setiap truk digambar pada subplot tersendiri.

2.8 Implementasi Bonus Objective Function

Bonus ini meminta objective function selain yang telah didefinisikan tanpa menambah variabel yang ada. Kelompok kami membuat dua objective function baru, yaitu `value_per_weight` dan `utilisasi`, di samping objective awal value (total value paket di dalam truk). Seluruh implementasinya terdapat pada file objective.py.

Objective `value_per_weight` menilai seberapa hemat berat sebuah susunan paket dalam menghasilkan nilai. Fungsi ini menghitung total value dikalikan kepadatan nilai, atau sama dengan kuadrat total value dibagi berat terpakai. Rasio murni (total value dibagi berat) tidak dipakai karena rasio tersebut mendorong pencarian menyisakan satu paket dengan rasio terbaik saja, sehingga truk hampir kosong. Pada bentuk kuadrat, penambahan paket masih menguntungkan walaupun kepadatan nilai paket tersebut lebih rendah daripada separuh kepadatan nilai saat ini. Akibatnya, truk tetap terisi, tetapi paket yang boros berat cenderung tersingkir. Untuk truk kosong, fungsi mengembalikan 0, dan pembagian bulat dipakai agar hasilnya bilangan bulat.

Objective `utilisasi` menilai total value dengan bonus pemanfaatan ruang dan berat. Fungsi ini mengalikan total value dengan faktor $1 + 0{,}5 \times (\text{ruang} + \text{berat})$, dengan ruang adalah volume terpakai dibagi volume seluruh truk dan berat adalah berat terpakai dibagi kapasitas seluruh truk. Faktor tersebut berkisar antara 1 dan 2, sehingga total value tetap menjadi penentu utama, sedangkan pemanfaatan ruang dan berat berperan sebagai bonus. Karena perhitungan memakai total semua truk, fungsi ini juga berlaku untuk kasus multitruk. Hasilnya dibulatkan ke bilangan bulat agar aman untuk perbandingan kesamaan nilai pada Sideways Move dan untuk seleksi roulette wheel pada Genetic Algorithm.

```
def objective_value(state: State):
    return state.total_value


def objective_value_per_weight(state: State):
    weight = state.current_weight
    if weight == 0:
        return 0
    return state.total_value ** 2 // weight
```

```
def objective_utilisasi(state: State):
   ruang = state.current_volume / state.total_volume if
state.total_volume else 0
   berat = state.current_weight / state.total_capacity if
state.total_capacity else 0
   return round(state.total_value * (1 + 0.5 * (ruang +
berat)))
```

Selanjutnya, ketiga objective diintegrasikan dalam satu program sehingga dapat dipilih saat program berjalan. Fungsi objective bertindak sebagai pemilih: fungsi ini mengembalikan `INVALID_VALUE (-999)` untuk state yang tidak valid, lalu menghitung nilai sesuai mode aktif untuk state yang valid. Karena Hill-Climbing, Simulated Annealing, dan Genetic Algorithm sama-sama memanggil objective(state), ketiganya tidak perlu diubah. Pengguna memilih mode melalui menu "Ganti objective function" pada main.py. Hasil eksperimen dengan objective bukan default disimpan di folder terpisah yang diberi akhiran nama objective, misalnya `output/ga/dataset_n30_utilisasi` dan `output/ga/dataset_n30_value_per_weight`, sedangkan objective default tetap memakai nama lama. Selain itu, berkas summary.csv mencatat nama objective dan total value asli state akhir, sehingga hasil antar-*objective* tetap dapat dibandingkan walaupun skala nilainya berbeda.

2.9 Implementasi Visualisasi

2.10 Hasil Eksperimen dan Analisis

2.10.1 Pengaturan Eksperimen

Eksperimen dilakukan menggunakan dua *file input.* File `dataset_n30.json` berisi 30 paket dengan satu truk berukuran 10 x 20 x 5 dan kapasitas 500. File `dataset_n30_multitruck.json` berisi 30 paket yang sama, dengan truk pertama 10 × 20 × 5 berkapasitas 500 dan truk kedua 8 × 12 × 5 berkapasitas 300. Total *value* seluruh paket adalah 7735 dan total beratnya 922 sehingga tidak semua paket dapat diangkut. Hill-Climbing dan Simulated Annealing dijalankan tiga kali dengan seed 1, 2, dan 3 sehingga algoritma tersebut dimulai dari *initial state* yang sama pada *run* yang sama. Genetic Algorithm dijalankan tiga kali untuk setiap konfigurasi dengan seed 1000–1017.

2.10.2 Hasil Hill-Climbing

Berikut adalah hasil eksperimen menggunakan algoritma Hill Climbing dengan berbagai variasi pada `dataset_n30.json`.

Steepest Ascent

`max_iteration = 1000, num_neighbor = 20`

Run	Seed	Initial Value	Final Value	Selisih	Iterasi	Durasi (s)

1	1	1511	2227	+716	5	0.0361
2	2	1653	1903	+250	2	0.0128
3	3	1797	2747	+950	11	0.1017

Sideways Move

`max_iteration = 1000, num_neighbor = 20, max_sideways_move = 100`

Run	Seed	Nilai awal	Nilai akhir	Selisih	Iterasi	Sideways move	Langkah Perbaikan	Durasi (s)
1	1	1511	4094	+2583	220	196	24	4,3688
2	2	1653	4242	+2589	353	321	32	6,0213
3	3	1797	4591	+2794	420	386	34	7,2839

Random Restart

`num_neighbor = 20, max_restart = 5`

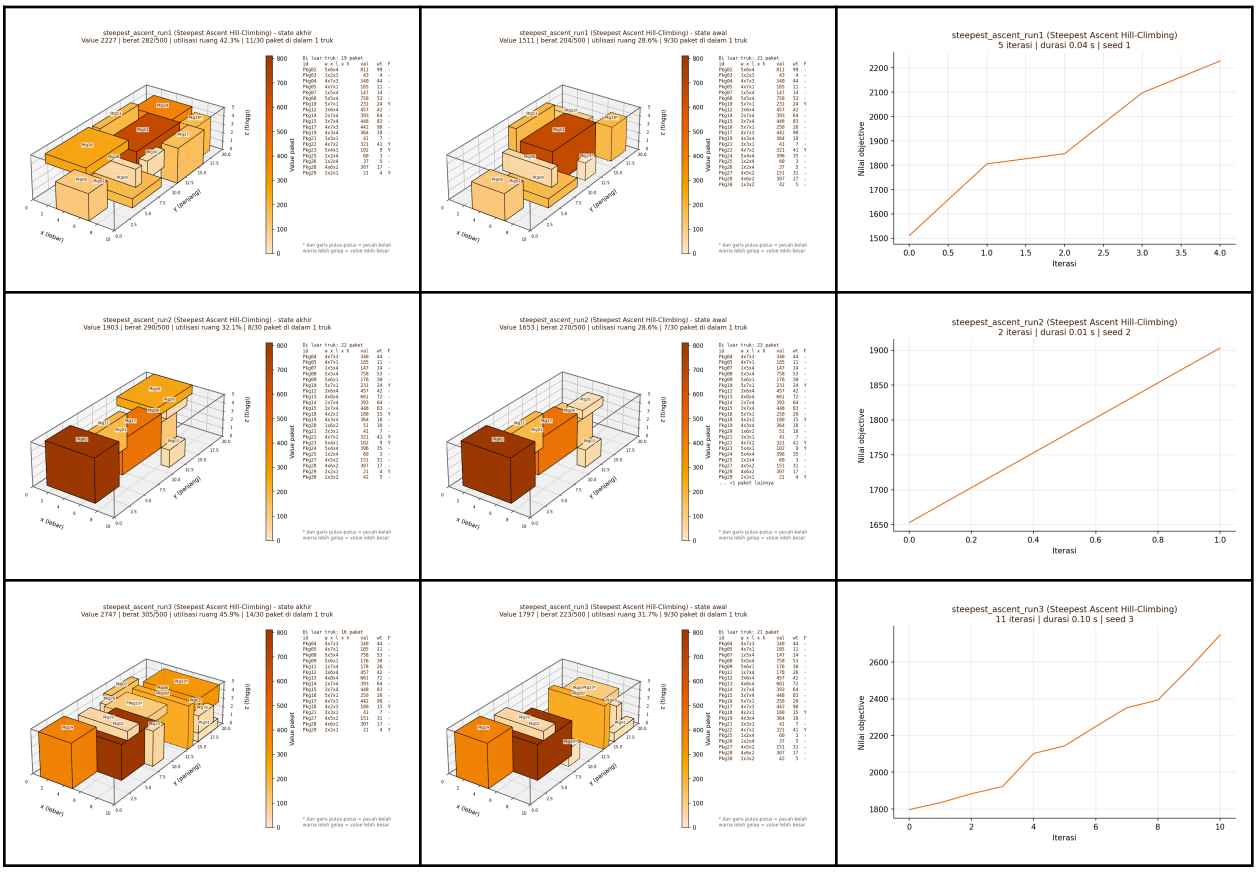
Run	Seed	Initial Value	Final Value	Selisih	Banyak restart	Iterasi	Durasi (s)
1	1	1511	2428	+917	5	44	0.2817
2	2	1653	3378	+1725	5	61	0.2881
3	3	1797	3047	+1250	5	44	0.2176

Stochastic

`max_iteration = 1000`

Run	Seed	Initial Value	Final Value	Selisih	Iterasi	Durasi (s)
1	1	1511	3212	+1701	1000	0,6851
2	2	1653	3339	+1686	1000	0,6084
3	3	1797	3094	+1297	1000	0,5823

Berikut visualisasi data hasil pengujian untuk setiap algoritma Hill Climbing untuk algoritma Steepest Ascent (untuk visualisasi algoritma yang lain dapat dilihat pada folder output program).

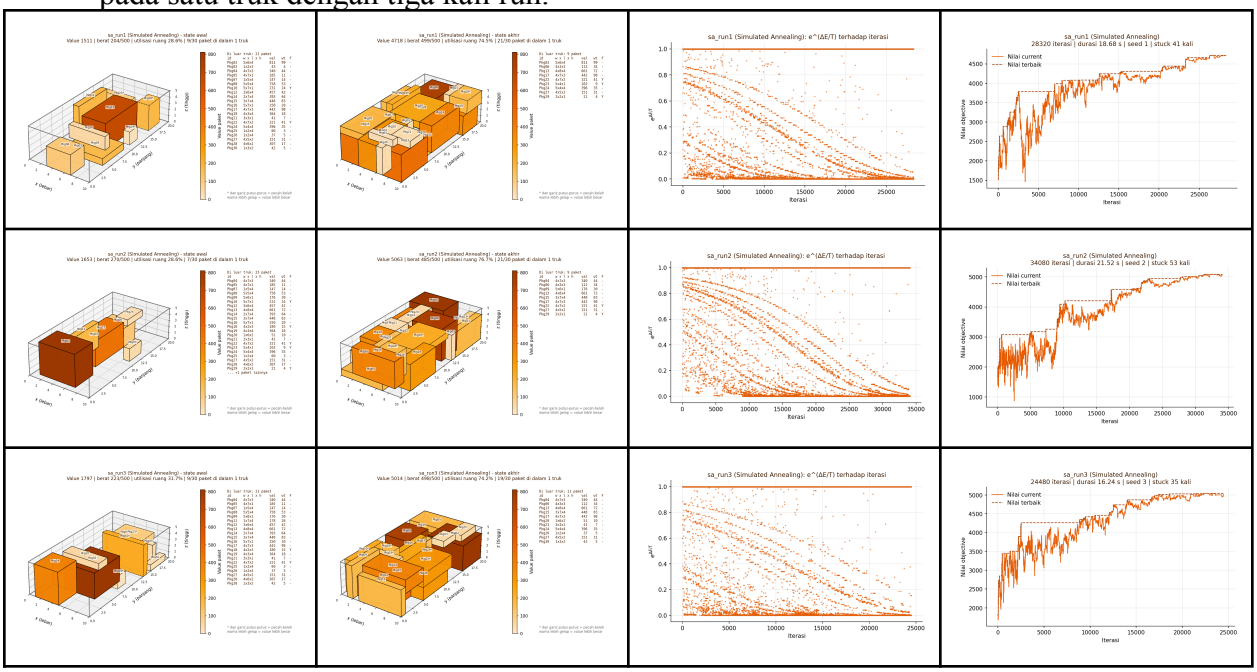


2.10.3 Hasil Simulated Annealing

Berikut terlampir hasil algoritma Simulated Annealing yang dijalankan pada `dataset_n30.json` (L = 16 × 30 = 480 iterasi per level temperatur).

Run	*Seed*	Nilai Awal	Nilai Akhir	Nilai Terbaik	Iterasi	Level T	Frek. *Stuck*	T_0	Durasi (s)	Paket di truk	Berat
1	1	1511	4718	4718	28320	59	41	143,65	18,68	21	499 /500
2	2	1653	5063	5084	34080	71	53	262,03	21,52	21	485 /500
3	3	1797	5014	5042	24480	51	35	147,06	16,24	19	498 /500
Rata-rata			4931,67 152,40)						18,81		

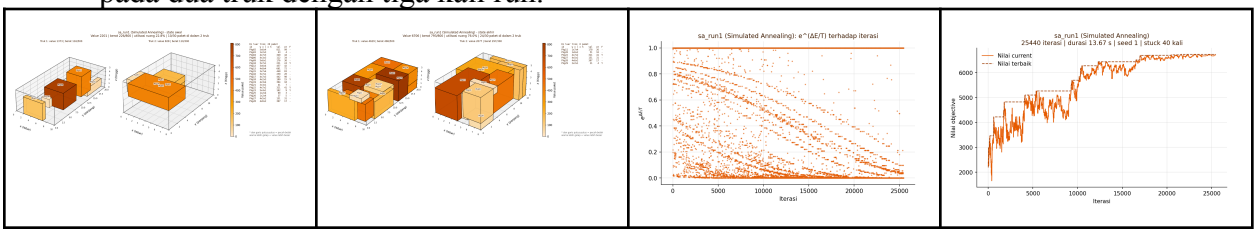
Berikut visualisasi data hasil pengujian untuk algoritma Simulated Annealing pada satu truk dengan tiga kali run.

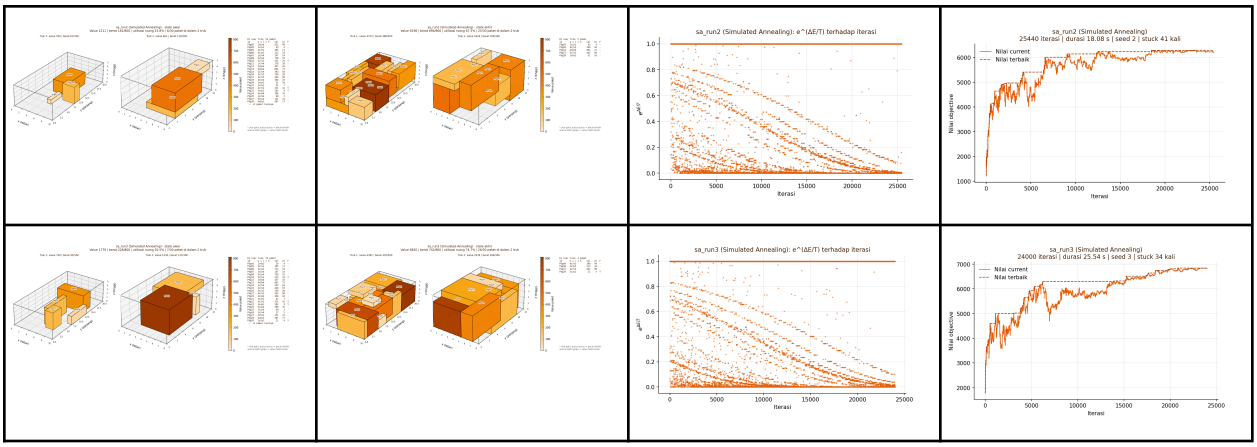


Berikut terlampir juga hasil algoritma Simulated Annealing yang dijalankan pada `dataset_n30_multitruck.json`.

Run	*Seed*	Nilai Awal	Nilai Akhir	Nilai Terbaik	Iterasi	Level T	Frek. *Stuck*	T_0	Durasi (s)	Paket (T1+T2)	Berat (T1+T2)
1	1	2201	6706	6729	25440	53	40	185,47	13,67	14+10	496+ 297
2	2	1211	6198	6292	25440	53	41	117,92	18,08	16+9	486+ 208
3	3	1779	6820	6841	24000	50	34	113,44	25,54	17+9	476+ 266
Rata-rata			6574,67 270,38)						19,10		

Berikut visualisasi data hasil pengujian untuk algoritma Simulated Annealing pada dua truk dengan tiga kali run.



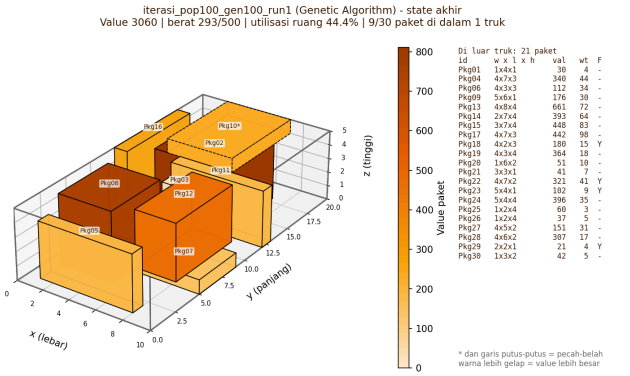


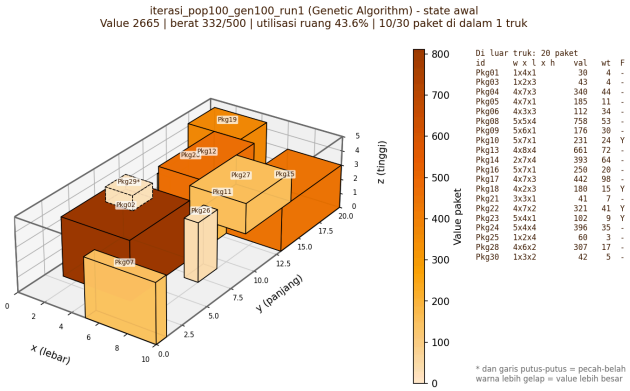
2.10.4 Hasil Genetic Algorithm

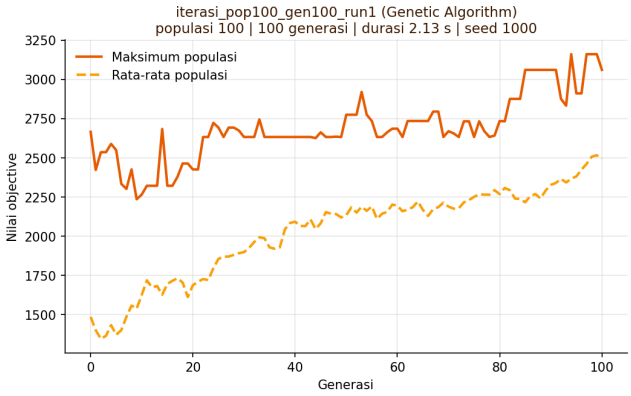
Berikut terlampir hasil Genetic Algorithm yang dijalankan pada dataset_n30.json (mutation rate 0,1, tanpa elitisme).

Variasi	Populasi	Generasi	Nilai akhir run 1 / 2 / 3	Rata-rata (simpangan baku)	Rata-rata durasi (s)
Iterasi	100	100	3060 / 2975 / 2744	2926,33 (133,52)	2,62
Iterasi	100	300	3345 / 3971 / 3669	3661,67 (255,62)	9,98
Iterasi	100	500	3041 / 4298 / 3265	3534,67 (547,45)	15,60
Populasi	50	300	3073 / 3155 / 3456	3228,00 (164,66)	4,38
Populasi	100	300	3273 / 3577 / 3720	3523,33 (186,39)	8,58
Populasi	200	300	3485 / 3654 / 3688	3609,00 (88,77)	15,34

Adapun contoh visualisasi paket satu truk untuk run 1 dengan 100 populasi dan 100 generasi sebagai berikut.





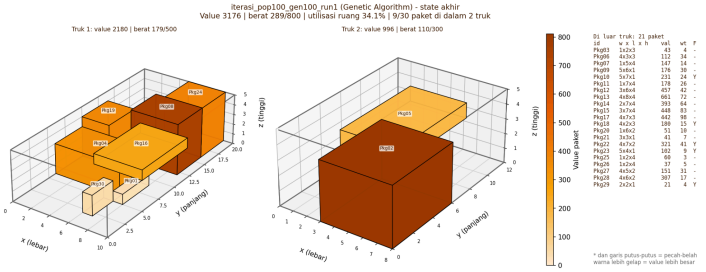


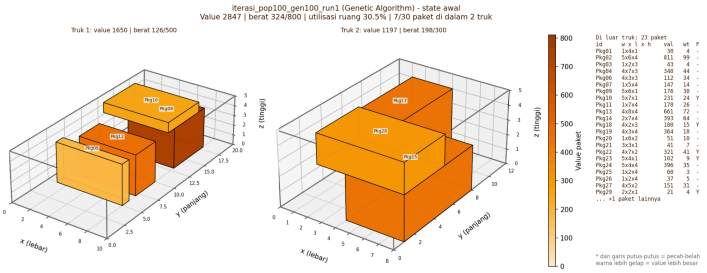
Berikut terlampir hasil Genetic Algorithm yang dijalankan pada `dataset n30 multitruck.json`.

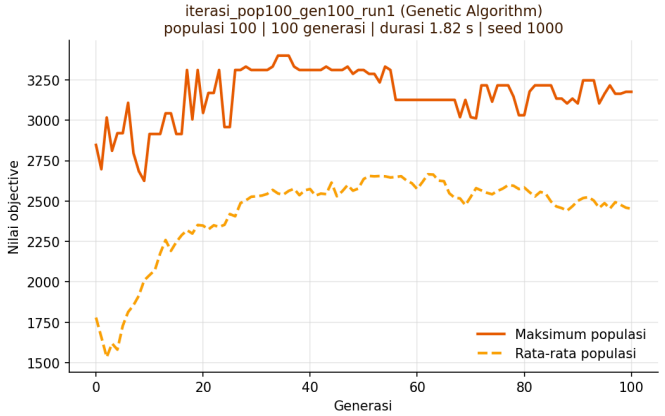
Variasi	Populasi	Generasi	Nilai akhir run 1 / 2 / 3	Rata-rata (simpangan baku)	Rata-rata durasi (s)
Iterasi	100	100	3176 / 3551 / 3708	3478,33 (223,18)	3,52
Iterasi	100	300	3826 / 4053 /	4061,33 (195,64)	7,58

			4305		
Iterasi	100	500	4418 / 4231 / 4438	4362,33 (93,22)	11,63
Populasi	50	300	3489 / 3642 / 4467	3866,00 (429,54)	3,31
Populasi	100	300	4267 / 3700 / 3827	3931,33 (242,95)	7,31
Populasi	200	300	4482 / 4898 / 4712	4697,33 (170,15)	17,40

Adapun contoh visualisasi paket untuk run 1 dengan 100 populasi dan 100 generasi sebagai berikut.



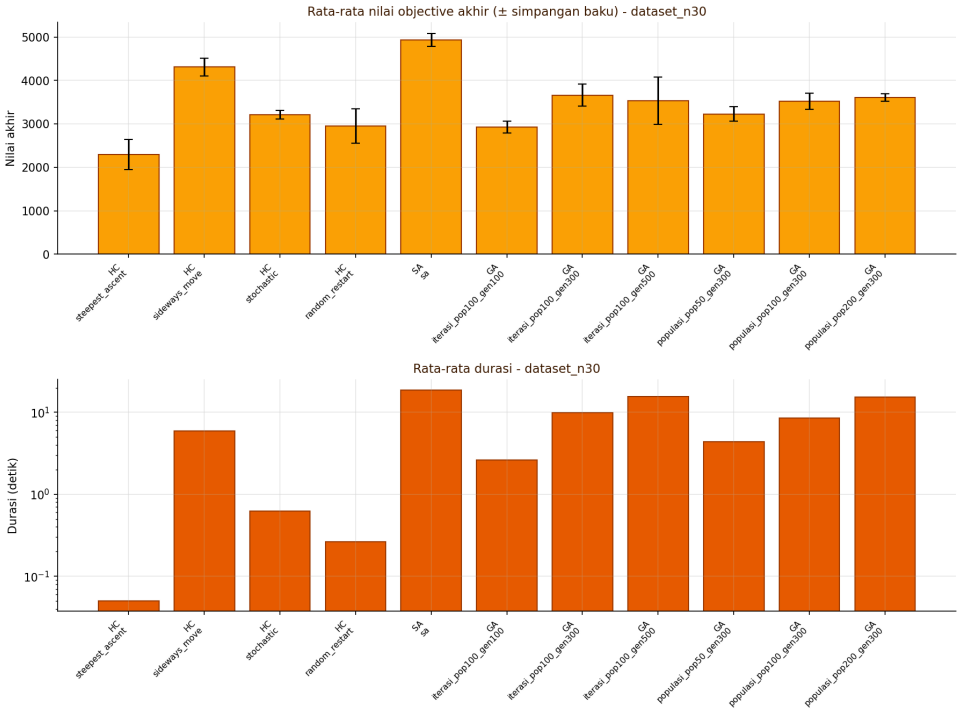




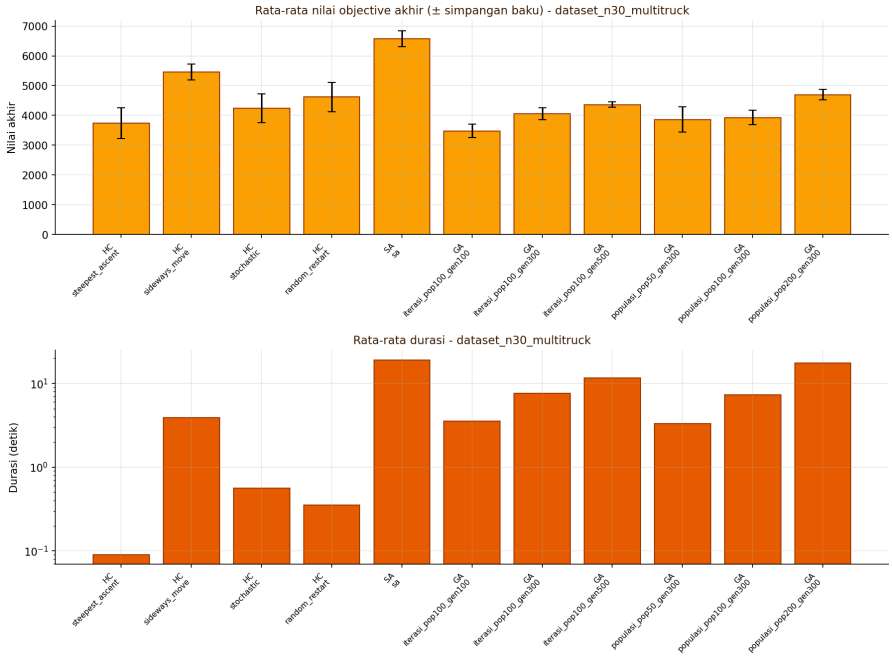
2.9.5 Analisis Perbandingan Antaralgoritma

Perbandingan hasil keenam algoritma pada kedua input dirangkum pada tabel di bawah ini, sedangkan rata-rata nilai akhir dan durasi tiap algoritma divisualisasikan pada kedua gambar di bawah tabel. Nilai yang dibandingkan adalah rata-rata dari tiga run. Untuk Genetic Algorithm, nilai yang digunakan berasal dari konfigurasi dengan rata-rata nilai akhir tertinggi.

Algoritma	(n30) Rata-rata akhir (simpangan baku)	% Batas atas	Durasi (s)	(Multi-truk) Rata-rata akhir (simpangan baku)	% Batas atas	Durasi (s)
HC Steepest Ascent	2292,33 (347,64)	42,2%	0,05	3740,33 (520,69)	52,1%	0,09
HC Sideways Move	4309,00 (208,36)	79,4%	5,89	5465,00 (265,56)	76,1%	3,90
HC Stochastic	3215,00 (100,04)	59,3%	0,63	4238,67 (488,56)	59,0%	0,56
HC Random Restart	2951,00 (393,73)	54,4%	0,26	4617,00 (484,12)	64,3%	0,35
Simulated Annealing	4931,67 (152,40)	90,9%	18,81	6574,67 (270,38)	91,6%	19,10
Genetic Algorithm	3661,67 (255,62)	67,5%	9,98	4697,33 (170,15)	65,4%	17,40



Gambar Perbandingan Eksperimen Satu Truk



Gambar Perbandingan Eksperimen Multi-truk

Berdasarkan tabel tersebut, Simulated Annealing menghasilkan rata-rata nilai akhir tertinggi pada kedua input, yaitu 4931,67 pada kasus satu truk dan 6574,67 pada kasus dua truk. Nilai tersebut setara dengan sekitar 91% dari batas atas nilai optimal. Posisi kedua ditempati oleh Hill-Climbing Sideways Move dengan 76–79% dari batas atas. Urutan kinerja seluruh algoritma sama pada kasus satu truk maupun dua truk. Hal

ini menunjukkan bahwa perbedaan kinerja tersebut konsisten dan tidak hanya berlaku untuk satu input tertentu.

Ruang pencarian pada persoalan ini didominasi oleh plateau, yaitu kumpulan *state* dengan nilai *objective* yang sama. Hal ini terjadi karena sebagian besar neighbor tidak mengubah total *value*, misalnya hasil rotasi paket atau pergeseran paket di dalam truk. Kondisi tersebut paling berpengaruh pada Steepest Ascent Hill-Climbing yang berhenti setelah hanya 2 sampai 16 iterasi, karena tidak ada neighbor yang lebih baik di antara 20 neighbor yang dibangkitkan. Padahal, *state* tempat pencarian berhenti belum tentu merupakan *local maximum* yang sebenarnya.

Sebaliknya, Hill-Climbing Sideways Move dapat terus bergerak di sepanjang plateau. Sekitar 86–92% iterasi pada Hill-Climbing Sideways Move merupakan *sideways move*, dan pada kasus satu truk rata-rata nilai akhirnya mencapai hampir dua kali nilai Steepest Ascent. Simulated Annealing juga dapat melintasi plateau karena langkah dengan $\Delta E = 0$ selalu diterima. Selain itu, Simulated Annealing juga dapat menerima langkah yang memperburuk dengan peluang e^(ΔE/T) sehingga mampu keluar dari *local maximum* yang tidak dapat dilewati oleh Sideways Move. Kemampuan tersebut yang membuat Simulated Annealing menghasilkan nilai tertinggi.

Stochastic Hill-Climbing tidak berhenti ketika berada di plateau, karena pencarian terus berjalan hingga batas 1000 iterasi sehingga hasilnya lebih baik daripada Steepest Ascent. Meskipun demikian, algoritma ini hanya menerima neighbor yang nilainya lebih besar sehingga tetap tidak dapat keluar dari *local maximum*. Random Restart Hill-Climbing menaikkan hasil Steepest Ascent sebesar 659 pada kasus satu truk dan 877 pada kasus dua truk. Kenaikan ini diperoleh dengan mengambil hasil terbaik dari enam kali pencarian yaitu satu pencarian awal dan lima kali *restart*. Namun, karena setiap pencarian Steepest Ascent berlangsung sangat singkat, hasil Random Restart tetap berada jauh di bawah Simulated Annealing.

Nilai awal yang digunakan oleh Genetic Algorithm relatif tinggi, yaitu sekitar 2400 sampai 3300, karena nilai awal yang dicatat adalah individu terbaik dari 50 sampai 200 *state* acak pada populasi awal. Akan tetapi, peningkatan nilai selama proses evolusi relatif kecil. *Crossover* satu titik pada persoalan ini sering menghasilkan individu anak dengan paket yang saling tumpang tindih atau melayang sehingga fungsi *repair* harus mengeluarkan sebagian paket dari truk. Akibatnya, individu terbaik pada generasi terakhir hanya memuat sekitar 8 sampai 17 paket dengan total berat yang masih jauh di bawah kapasitas truk. Selain itu, karena implementasi tidak menggunakan elitisme, individu terbaik pada suatu generasi dapat hilang pada generasi berikutnya.

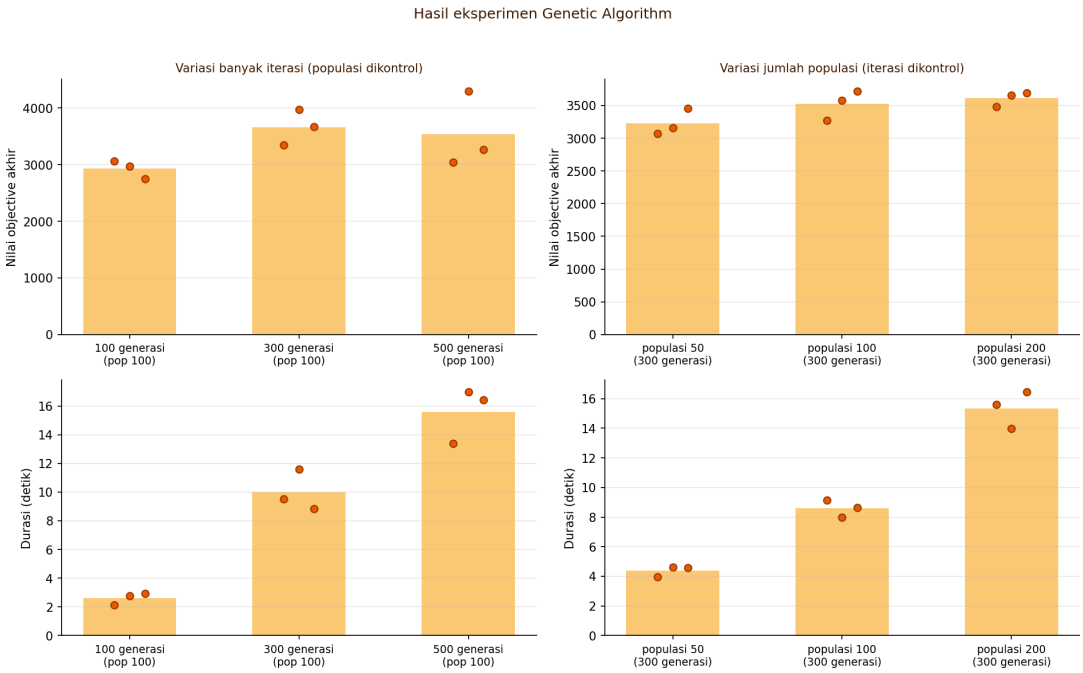
Pada implementasi ini, konsistensi hasil diukur dari simpangan baku nilai akhir pada tiga *run*. Simulated Annealing memiliki simpangan baku 152,40 pada kasus satu truk, lebih kecil daripada Sideways Move (208,36) sehingga hasilnya cukup stabil. Steepest Ascent dan Random Restart memiliki simpangan baku yang relatif besar (347,64 sampai 520,69) karena hasilnya sangat bergantung pada *initial state*. Stochastic Hill-Climbing memiliki simpangan baku terkecil pada kasus satu truk (100,04), tetapi nilai akhirnya tergolong rendah.

Varian Hill-Climbing merupakan algoritma tercepat dengan rata-rata durasi 0,05 sampai 5,89 detik, sedangkan Simulated Annealing menjadi yang paling lama dengan rata-rata durasi sekitar 19 detik, karena menjalankan 24.000 sampai 34.000 iterasi hingga kondisi *frozen* tercapai. Durasi Genetic Algorithm meningkat seiring bertambahnya

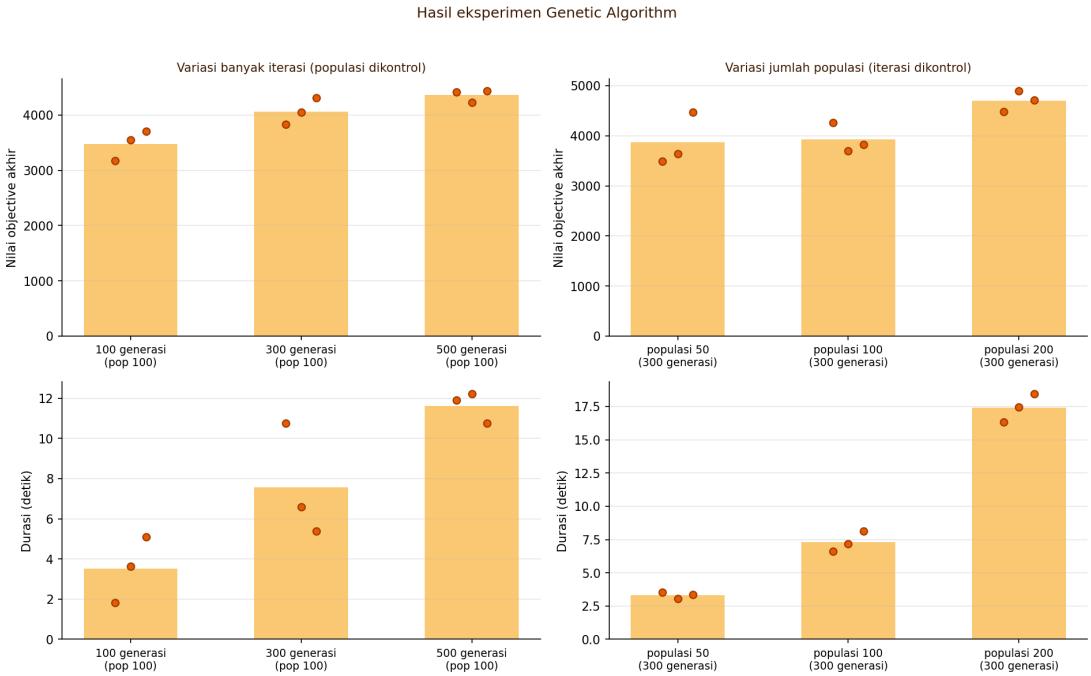
jumlah populasi dan generasi karena jumlah individu yang dievaluasi sebanding dengan hasil kali keduanya.

Untuk kasus multi-truk, penambahan truk kedua meningkatkan nilai akhir pada seluruh algoritma karena total kapasitas angkut bertambah dari 500 menjadi 800. Simulated Annealing dan Sideways Move mampu memanfaatkan kedua truk dengan baik. Contohnya, Simulated Annealing mengisi kedua truk dengan total berat 694 sampai 793 dari kapasitas 800. Sebaliknya, Genetic Algorithm cenderung hanya mengisi truk kedua dengan sedikit paket sehingga kenaikan nilainya lebih kecil.

2.9.6 Analisis Pengaruh Iterasi dan Populasi Genetic Algorithm



Gambar Hasil Eksperimen Genetic Algorithm Satu Truk



Gambar Hasil Eksperimen Genetic Algorithm Multitruk

Eksperimen Genetic Algorithm menguji pengaruh dua parameter, yaitu banyak iterasi (generasi) dan jumlah populasi, terhadap nilai akhir pencarian. Pada eksperimen pertama, jumlah populasi dijaga tetap 100 sementara banyak iterasi divariasikan menjadi 100, 300, dan 500. Pada eksperimen kedua, banyak iterasi dijaga tetap 300 sementara jumlah populasi divariasikan menjadi 50, 100, dan 200. Setiap konfigurasi dijalankan tiga kali pada dua dataset, yaitu `dataset_n30` (satu truk) dan `dataset_n30_multitruck`.

Pada dataset satu truk, penambahan iterasi dari 100 ke 300 generasi meningkatkan rata-rata nilai akhir dari 2926 menjadi 3662, atau sekitar 25%. Ketiga run pada 300 generasi (3345 sampai 3971) lebih baik daripada ketiga run pada 100 generasi (2744 sampai 3060). Namun, penambahan iterasi ke 500 generasi tidak memperbaiki hasil secara jelas. Rata-rata nilai akhir turun menjadi 3535, dan sebaran antar-run melebar (simpangan baku 547). Satu run memang mencapai 4298, yaitu nilai tertinggi pada dataset ini, tetapi dua run lainnya (3041 dan 3265) berada di bawah rata-rata 300 generasi. Dengan demikian, manfaat tambahan iterasi pada dataset satu truk tampak berkurang setelah sekitar 300 generasi.

Sebaliknya, pada dataset multitruk, penambahan iterasi terus memperbaiki hasil. Rata-rata nilai akhir naik dari 3478 (100 generasi) ke 4061 (300 generasi), lalu ke 4362 (500 generasi). Kenaikan tahap pertama sekitar 17%, sedangkan kenaikan tahap kedua sekitar 7%, sehingga manfaat tambahan iterasi tetap mengecil. Selain itu, hasil antar-run semakin konsisten, ditunjukkan simpangan baku yang turun dari 223 menjadi 196 lalu 93. Meskipun begitu, rentang hasil 300 dan 500 generasi masih sedikit tumpang tindih karena nilai tertinggi pada 300 generasi (4305) lebih besar daripada nilai terendah pada 500 generasi (4231).

Selanjutnya, analisis jumlah populasi. Pada dataset satu truk, rata-rata nilai akhir naik dari 3228 (populasi 50) ke 3523 (populasi 100), lalu ke 3609 (populasi 200). Kenaikan dari populasi 50 ke 100 sekitar 9%, sedangkan kenaikan dari populasi 100 ke

200 hanya sekitar 2,4%. Populasi 200 juga paling konsisten dengan simpangan baku 89, dan nilai terendahnya (3485) lebih tinggi daripada nilai terendah populasi 50 (3073) dan populasi 100 (3273). Populasi besar membantu karena populasi tersebut memuat lebih banyak susunan awal yang beragam, sehingga individu awal terbaiknya pun lebih baik (rata-rata 2441, 2589, dan 2995).

Pada dataset multitruk, pengaruh populasi terlihat berbeda. Populasi 50 dan 100 menghasilkan rata-rata yang hampir sama, yaitu 3866 dan 3931, dan selisih 65 itu lebih kecil daripada sebaran antar-run (simpangan baku 430 dan 243). Sebaliknya, populasi 200 menghasilkan lonjakan yang jelas dengan rata-rata 4697, atau sekitar 19,5% lebih tinggi daripada populasi 100. Semua run populasi 200 (4482 sampai 4898) juga lebih baik daripada semua run populasi 100 (3700 sampai 4267). Selain itu, populasi 200 dengan 300 generasi mengevaluasi sekitar 60.000 individu, hampir sebanding dengan populasi 100 dengan 500 generasi yang mengevaluasi sekitar 50.000 individu. Pada beban yang mirip itu, populasi besar menghasilkan rata-rata lebih tinggi (4697 dibanding 4362). Oleh karena itu, pada dataset multitruk, memperbesar populasi cenderung lebih efektif daripada menambah generasi.

Di samping nilai akhir, kedua parameter juga memengaruhi durasi pencarian. Pada dataset satu truk, rata-rata durasi naik dari 2,6 detik (100 generasi) ke 10,0 detik (300 generasi) dan 15,6 detik (500 generasi), sedangkan untuk populasi 50, 100, dan 200 durasinya 4,4, 8,6, dan 15,3 detik. Pada dataset multitruk, durasi untuk 100, 300, dan 500 generasi adalah 3,5, 7,6, dan 11,6 detik, sedangkan untuk populasi 50, 100, dan 200 adalah 3,3, 7,3, dan 17,4 detik. Dengan kata lain, durasi naik kira-kira sebanding dengan jumlah individu yang dievaluasi. Akibatnya, konfigurasi dengan hasil terbaik selalu menjadi konfigurasi yang paling lama dijalankan.

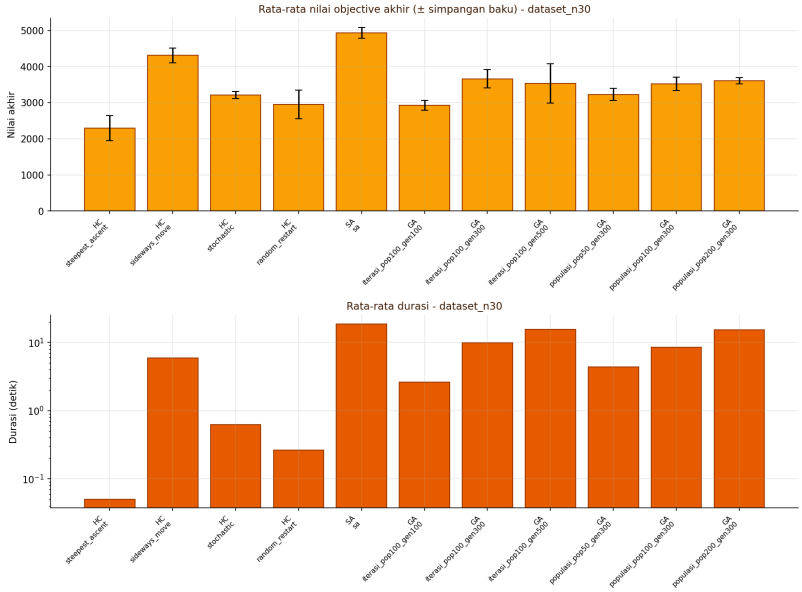
Meskipun demikian, kesimpulan di atas perlu dibaca dengan hati-hati. Setiap konfigurasi hanya dijalankan tiga kali, sehingga perbedaan kecil antarkonfigurasi belum tentu bermakna. Konfigurasi populasi 100 dengan 300 generasi dijalankan dua kali dengan *seed* berbeda, dan hasilnya berselisih sekitar 138 pada dataset satu truk (3662 dan 3523) serta sekitar 130 pada dataset multitruk (4061 dan 3931). Oleh sebab itu, selisih sekecil itu, misalnya antara 300 dan 500 generasi pada dataset satu truk, tidak dapat dianggap perbedaan nyata. Durasi pun bervariasi antar-*run* pada konfigurasi yang sama, misalnya 5,4 sampai 10,8 detik pada populasi 100 dengan 300 generasi di dataset multitruk, kemungkinan karena beban mesin saat eksperimen berlangsung.

Dengan demikian, banyak iterasi dan jumlah populasi sama-sama meningkatkan hasil akhir Genetic Algorithm, tetapi keuntungannya semakin kecil sementara durasi terus bertambah. Pada dataset satu truk, kombinasi populasi 100 dengan 300 generasi memberi keseimbangan terbaik antara hasil dan waktu. Pada dataset multitruk, hasil masih terus membaik pada iterasi dan populasi yang lebih besar, terutama pada populasi 200. Perbedaan pola ini diduga muncul karena ruang pencarian pada multitruk lebih besar, sehingga pencarian membutuhkan lebih banyak iterasi dan variasi populasi sebelum hasilnya mendatar.

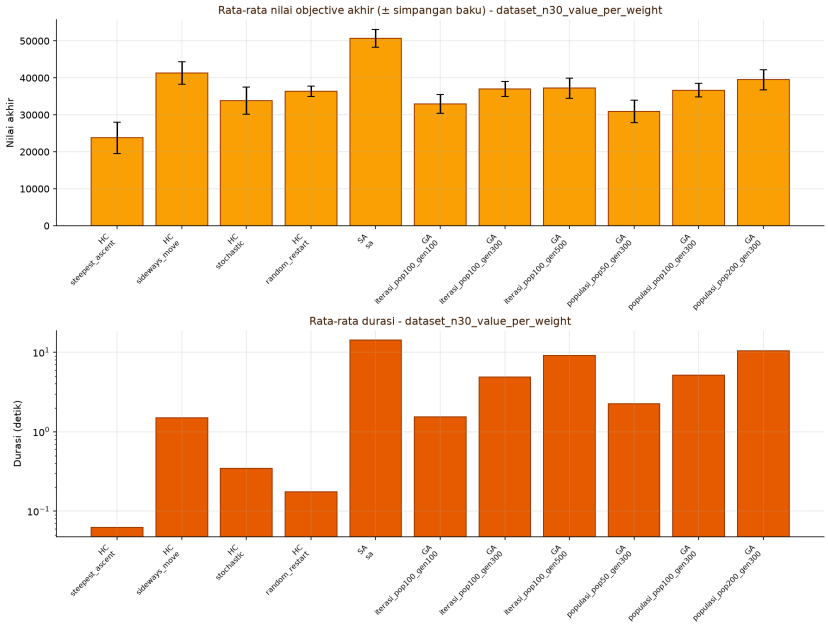
2.9.7 Analisis Objective Function Utilitas dan Value per Weight

Analisis ini membandingkan tiga objective function pada dataset dataset_n30, yaitu value (total value paket di dalam truk), value_per_weight (total value dikalikan kepadatan nilai, atau total value kuadrat dibagi berat terpakai), dan utilisasi (total value

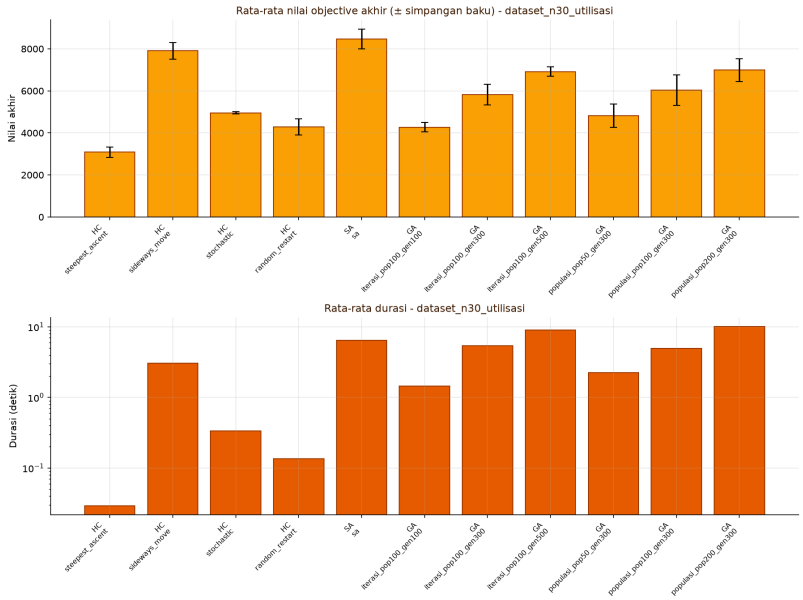
dikalikan faktor 1 sampai 2 yang naik seiring pemanfaatan ruang dan berat). Ketiga objective menghasilkan nilai dengan skala yang jauh berbeda, yaitu sekitar 2.000 sampai 5.000 untuk value, 3.000 sampai 8.500 untuk utilisasi, dan 24.000 sampai 51.000 untuk value_per_weight. Karena itu, tinggi batang pada ketiga grafik tidak dapat dibandingkan secara langsung antar-objective.



Gambar Objective Function Value



Gambar Objective Function Value per Weight



Gambar Objective Function Utilisasi

Untuk mengatasi perbedaan skala tersebut, analisis ini memakai dua perhitungan. Pertama, skor relatif setiap algoritma dihitung sebagai nilai rata-rata algoritma dibagi nilai rata-rata Simulated Annealing (SA) pada objective yang sama, lalu dikalikan 100%. SA dipilih sebagai pembanding karena SA memiliki nilai tertinggi pada ketiga objective, sehingga skornya selalu 100%. Kedua, kenaikan relatif pada Genetic Algorithm (GA) dihitung sebagai (nilai konfigurasi besar dikurangi nilai konfigurasi kecil) dibagi nilai konfigurasi kecil. Nilai rata-rata GA pada objective value diambil langsung dari summary.csv. Nilai lainnya dibaca dari tinggi batang pada grafik perbandingan sehingga berupa perkiraan dengan ketelitian sekitar 3%.

Algoritma (varian)	**Value**	**Skor relatif**	**Utilisasi**	**Skor relatif**	**Value per weight**	**Skor relatif**
HC steepest ascent	2.290	46,5%	3.090	36,5%	23.800	46,9%
HC sideways move	4.310	87,4%	7.900	93,3%	41.300	81,5%
HC stochastic	3.200	64,9%	4.950	58,4%	33.800	66,7%
HC random restart	2.940	59,6%	4.280	50,5%	36.300	71,6%
SA	4.930	100%	8.470	100%	50.700	100%
GA, 100	2.926	59,4%	4.260	50,3%	32.900	64,9%

generasi (pop 100)						
GA, 300 generasi (pop 100)	3.662	74,3%	5.830	68,8%	37.000	73,0%
GA, 500 generasi (pop 100)	3.535	71,7%	6.910	81,6%	37.200	73,4%
GA, populasi 50 (300 generasi)	3.228	65,5%	4.820	56,9%	30.900	60,9%
GA, populasi 100 (300 generasi)	3.523	71,5%	6.030	71,2%	36.600	72,2%
GA, populasi 200 (300 generasi)	3.609	73,2%	7.000	82,6%	39.500	77,9%

Hasil pertama menunjukkan bahwa urutan algoritma pada posisi teratas dan terbawah tidak berubah antar-objective. SA menempati posisi pertama pada ketiga objective, HC *sideways move* menempati posisi kedua dengan skor relatif 81% sampai 93%, dan HC *steepest ascent* menempati posisi terakhir dengan skor 36% sampai 47%. Namun, keunggulan SA atas *sideways move* tidak sama jelasnya pada ketiga objective. Pada value dan value_per_weight, simpangan baku kedua algoritma tidak saling tumpang tindih, sedangkan pada utilisasi kedua simpangan baku saling tumpang tindih (sekitar 8.000 sampai 8.300). Dengan demikian, pada objective utilisasi, selisih SA dan *sideways move* belum dapat dianggap nyata.

Selanjutnya, posisi algoritma di tengah berubah mengikuti objective. HC *random restart* memiliki skor 59,6% pada value dan 50,5% pada utilisasi, tetapi naik menjadi 71,6% pada value_per_weight. HC *stochastic* relatif stabil di kisaran 58% sampai 67%. Perbedaan ini menunjukkan bahwa bentuk objective mempengaruhi seberapa baik setiap varian memanfaatkan perubahan kecil pada *state*. Meskipun begitu, penyebab pastinya belum dapat dipastikan karena setiap konfigurasi hanya dijalankan tiga kali.

Perubahan paling jelas terlihat pada GA. Pada objective utilisasi, kenaikan iterasi dari 100 ke 500 generasi menaikkan nilai sekitar 62% (4.260 menjadi 6.910), sedangkan pada value hanya sekitar 21% dan pada value_per_weight sekitar 13%. Pola yang sama muncul pada jumlah populasi. Kenaikan populasi dari 50 ke 200 menaikkan nilai sekitar 45% pada utilisasi, 28% pada value_per_weight, dan 12% pada value. Akibatnya, GA konfigurasi terbesar (populasi 200) mencapai 82,6% dari SA pada utilisasi, sedangkan pada value hanya 73,2%. Selain itu, pada value_per_weight, nilai GA dengan 300 dan 500 generasi hampir sama (37.000 dan 37.200), sehingga tambahan iterasi tidak memberi manfaat yang jelas.

Terakhir, kemungkinan penyebab GA lebih diuntungkan pada utilisasi adalah bentuk objective tersebut. Objective utilisasi mengalikan total value dengan faktor yang naik bersama pemanfaatan ruang dan berat, sehingga selisih nilai antar-individu menjadi lebih besar. Seleksi *roulette wheel* memilih orang tua sebanding dengan fitness, sehingga selisih yang lebih besar dapat membuat individu yang baik lebih sering terpilih. Penjelasan ini masih berupa dugaan karena belum diuji dengan data tambahan.

Mengenai durasi, nilai rata-rata SA dan HC *sideways move* tampak lebih singkat pada kedua objective baru. Durasi SA turun dari sekitar 18 detik pada value menjadi sekitar 6,5 detik pada utilisasi dan 14 detik pada *value_per_weight*, sedangkan durasi *sideways move* turun dari sekitar 6 detik menjadi 3,1 detik dan 1,5 detik. Namun, durasi dipengaruhi beban mesin saat eksperimen berjalan, sehingga selisih ini tidak dapat dikaitkan langsung dengan objective.

Meskipun begitu, analisis ini memiliki batasan yang jelas. Setiap konfigurasi hanya dijalankan tiga kali, dan banyak simpangan baku saling tumpang tindih, misalnya antara GA 300 dan 500 generasi pada value_per_weight. Grafik yang dipakai juga hanya memuat nilai objective, sehingga komposisi solusi tidak dapat dibandingkan. Total value asli pada objective baru, berat terpakai, ruang terpakai, dan jumlah paket tidak dapat dihitung dari grafik karena nilai utilisasi dan *value_per_weight* bergantung pada ketiga besaran tersebut. Oleh karena itu, klaim bahwa *value_per_weight* memilih paket yang lebih ringan atau bahwa utilisasi mengisi truk lebih penuh belum dapat dibuktikan.

Dengan demikian, perubahan objective function tidak mengubah urutan algoritma pada posisi teratas dan terbawah, tetapi mengubah seberapa besar GA diuntungkan oleh tambahan iterasi dan populasi. Pengaruh tersebut paling kuat pada objective utilisasi dan paling lemah pada value_per_weight. Pembuktian yang lebih kuat membutuhkan data komposisi solusi dan lebih banyak run per konfigurasi.

Bab 3
Kesimpulan dan Saran

3.1 Kesimpulan

Algoritma local search, yang meliputi Hill Climbing, Simulated Annealing, dan Genetic Algorithm berhasil diimplementasikan guna menyelesaikan permasalahan penyusunan paket-paket di dalam truk sesuai format data yang diberikan dan memenuhi constraint dengan implementasi melalui kode program Python.

Hasil eksperimen juga berhasil dilakukan untuk menguji beberapa test case untuk setiap algoritma guna mendapatkan pemahaman dan insight dari implementasi program. Pengujian juga dilengkapi visualisasi bagaimana penyusunan paket di dalam truk untuk mempermudah melakukan analisis.

Algoritma Hill Climbing dengan variasi Sideways Move dan Random Restart secara umum mampu memberikan hasil pencarian yang lebih baik karena cukup mampu menghindari local optima dibandingkan dengan algoritma Hill Climbing konvensional. Simulated Annealing juga berhasil mencatatkan durasi eksekusi yang efisien dan konvergensi cukup stabil. Sedangkan untuk algoritma Genetic Algorithm sangat efektif pada pengoptimalan objective function berbasis utilitas volume karena menggunakan roulette wheel selection dan crossover yang dapat memperbesar perbedaan fitness antar-individu meskipun membutuhkan waktu komputasi yang cukup tinggi.

3.2 Saran

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan, terdapat beberapa saran untuk pengembangan program selanjutnya.

1. Menambahkan elitisme pada Genetic Algorithm
2. Mengganti representasi kromosom pada Genetic Algorithm
3. Menggabungkan Simulated Annealing dengan Hill-Climbing

Pembagian Tugas

No	Nama / NIM	Deskripsi Tugas
1	Keisha Rizka Syofyani / 13524073	Membuat core program local search
		Membuat algoritma search Hill-Climbing utama, yaitu Sideways Move
		Mengerjakan bonus bagian implementasi seluruh algoritma Hill Climbing.
		Mengerjakan laporan : template laporan, deskripsi persoalan, pembahasan bagian representasi state, objective function dan constraint, algoritma Hill-Climbing, hasil eksperimen Hill-Climbing, kesimpulan, dan Form Penggunaan Generative AI
2	Vara Azzara Ramli Pulukadang / 13524091	Implementasi Genetic Algorithm
		Implementasi visualisasi
		Bonus Objective Function
		Mengerjakan laporan bagian Genetic Algorithm, visualialisasi, analisis objective function, Form Penggunaan Generative AI, Kesimpulan
3	Helena Kristela Sarhawa / 13524109	Membuat program bagian neighbor, experiment runner dan logger, README
		Membuat algoritma Simulated Annealing
		Mengerjakan bonus multi-truk
		Mengerjakan laporan bagian neighbor, Simulated Annealing, perbandingan ketiga algoritma, dan saran

Lampiran

1. **Repository Github**
   https://github.com/keirzka/IF3170_G19_lahtetibabanget

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2020. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk

[2] D. S. Johnson, C. R. Aragon, L. A. McGeoch, and C. Schevon, "Optimization by simulated annealing: An experimental evaluation; Part I, graph partitioning," *Operations Research*, vol. 37, no. 6, pp. 865–892, 1989.

[3] B. S. Baker, E. G. Coffman, Jr., and R. L. Rivest, "Orthogonal packings in two dimensions," SIAM J. Comput., vol. 9, no. 4, pp. 846–855, Nov. 1980.

[4] K. Karabulut and M. M. İnceoğlu, "A hybrid genetic algorithm for packing in 3D with deepest bottom left with fill method," in Advances in Information Systems (ADVIS 2004), Lecture Notes in Computer Science, vol. 3261. Berlin, Germany: Springer, 2004, pp. 441–450.

[5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020, ch. 4, sec. 4.1.

[6] W. Ben-Ameur, "Computing the initial temperature of simulated annealing," *Computational Optimization and Applications*, vol. 29, no. 3, pp. 369–385, Dec. 2004.

[7] Python Software Foundation, “Python 3 documentation.” [Online]. Available: https://docs.python.org/3/ (accessed Oct. 10, 2026).

Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Keisha Rizka Syofyani /13524073, Vara Azzara Ramli Pulukadang / 13524091, dan Helena Kristela Sarhawa / 13524109
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Laporan Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Menggunakan tools seperti Gemini dan Calude dalam memahami spesifikasi tugas dan konsep teori algoritma yang digunakan, serta memahami bug atau kesalahan dalam penulisan kode program
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026





Keisha Rizka Syofyani
13524073

Vara Azzara Ramli P
13524091

Helena Kristela Sarhawa
13524109

19